**Course**: Software Engineering (CMP-N204-0)

**Project:** Study-Buddy

**Group:** Imogene Garthwaite, Olivia Stars, Richard Akole, Sita Panda, Christy Francis.

## ***SPRINT 3:***

## Features for your application defined as user stories:

### Sprint 3 User Stories

**Register and Authentication:**

These stories describe how users join and access the system.

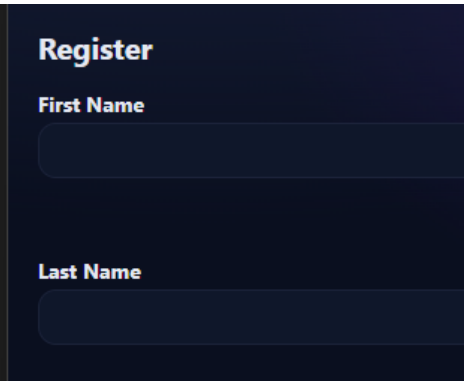
1. As a student, I want to create an account, so that I have access to the Study Buddy platform.

Front End:

```
form(method="POST" action="/register")
  div
    label(for="first_name") First Name
    br
    input(type="text" id="first_name" name="first_name" required)

  br

  div
    label(for="last_name") Last Name
    br
    input(type="text" id="last_name" name="last_name" required)
```




Back End:

```
app.post("/register", async (req, res) => {
  try {
    const { first_name, last_name, degree, email, password } = req.body;
  const emailRegex = /^[a-zA-Z0-9._%+-]+@roehampton\.ac\.uk$/;
```

2. As a student, when logging in, it should be secure to keep my personal information protected.

(Session management with a time out)

```
const hashedPassword = await bcrypt.hash(password, 10);
```

(Password hashing)

3. As a student, I should be able to reset my password if I forget it

```
router.post("/forgot-password", async (req, res) => {
  const { email } = req.body;
```

```
router.post("/reset-password", async (req, res) => {
  const { token, password } = req.body;
```

4. As a student I want the reassurance that my account will log out after a period of absence

```
// Sessions
app.use(session({
  secret: "studybuddy-secret",
  resave: false,
  saveUninitialized: false,
  rolling: true, // refresh expiry on activity
  cookie: { maxAge: 600000 } // 10 minutes
}));
```

**User Profiles:**

These stories are about how users establish their identities.

5. As a student, I should be able to create a profile with my course and modules to match other students easily.

```
await db.query(
    "INSERT INTO users (first_name, last_name, degree, email, password) VALUES (?, ?, ?, ?, ?)",
    [first_name, last_name, degree, email, hashedPassword]
);
```

(Profile creation)

```
degree VARCHAR(100),
```

(From SQL – database holds degree)

```
if (selectedDegree) {
  [users] = await db.query(
    "SELECT user_id, first_name, last_name, degree FROM users WHERE degree = ? ORDER BY first_name ASC",
    [selectedDegree]
```

(Users can be filtered/matched based on their course)

```
p
  strong Degree:
  |  #{user.degree}
```

(Pug showing this feature is available to users)

6. As a student, I want to edit my profile and update it to my study preferences.

```
// EDIT PROFILE PAGE (GET)
router.get('/edit', async (req, res) => {
    const user = req.session.user;
    res.render('edit-profile', { user });
});

// EDIT PROFILE (POST)
router.post('/edit', async (req, res) => {
    const userId = req.session.user.user_id;
    const { first_name, last_name, degree } = req.body;

    try {
        await req.db.query(
            "UPDATE users SET first_name = ?, last_name = ?, degree = ? WHERE user_id = ?",
            [first_name, last_name, degree, userId]
        );

        // Update session so profile updates instantly
        req.session.user.first_name = first_name;
        req.session.user.last_name = last_name;
        req.session.user.degree = degree;

        res.redirect('/profile');

    } catch (err) {
```

7. As a student profile, pictures can be uploaded, so other students can recognize me.

```
//  ADDED — UPLOAD PROFILE PICTURE PAGE (GET)
router.get('/upload-picture', (req, res) => {
    res.render('upload-picture');
});

//  ADDED — UPLOAD PROFILE PICTURE (POST)
router.post('/upload-picture', upload.single('profile_pic'), async (req, res) => {
    const userId = req.session.user.user_id;

    if (!req.file) {
        return res.status(400).send("No file uploaded.");
    }

    const filename = req.file.filename;

    await req.db.query(
        "UPDATE users SET profile_pic = ? WHERE user_id = ?",
        [filename, userId]
    );

    // Update session so profile updates instantly
    req.session.user.profile_pic = filename;

    res.redirect('/profile');
});
```

**Finding Study Partners:**

These stories are based on how users can search and discover their matches.

8. As a student, I am able to search for listings through the module to find others who study the same topic.

```
router.get("/:degree", async (req, res) => {
  try {
    const degree = decodeURIComponent(req.params.degree);

    const [rows] = await db.query(`
  SELECT
    listings.listing_id,
    listings.title,
    listings.location,
    listings.start_time,
    listings.status,
    modules.module_name,
    modules.level,
    tags.sessionType
  FROM listings
  JOIN modules ON listings.module = modules.module_name
  LEFT JOIN listing_tags ON listings.listing_id = listing_tags.listing_id
  LEFT JOIN tags ON listing_tags.tag_id = tags.tag_id
  WHERE modules.degree = ?
  ORDER BY listings.start_time ASC
```

9. As a student, I want to view other students' profiles to see if they are a good study match.

```
app.get("/users/:id", requireLogin, async (req, res) => {
  try {

    const userId = req.params.id;

    const [rows] = await db.query(
      "SELECT user_id, first_name, last_name, degree FROM users WHERE user_id = ?",
      [userId]
    );

    if (rows.length === 0) {
      return res.status(404).send("User not found.");
    }

    res.render("user-profile", {
      title: `${rows[0].first_name} ${rows[0].last_name}`,
      user: rows[0]
    });

  } catch (err) {
    console.error("User profile error:", err);
    res.status(500).send("Error loading user.");
  }
```

**Study Session Matching:**

These stories describe how users connect and plan a session.

10. As a student, I should be able to request a study session with another user to plan to revise together.

```
router.post("/:id/join", async (req, res) => {
  try {
    const userId = req.session.user.user_id;;
    const listingId = req.params.id;

    //  Prevent duplicate requests
    const [existing] = await db.query(
      "SELECT * FROM join_requests WHERE user_id = ? AND listing_id = ?",
      [userId, listingId]
    );

    if (existing.length > 0) {
      return res.redirect("/subjects?error=already_requested");
    }

    // Create request
    await db.query(
      "INSERT INTO join_requests (user_id, listing_id, status) VALUES (?, ?, 'pending')",
      [userId, listingId]
    );

    res.redirect("/subjects?success=request_sent");
```

11. As a student, I should be able to accept or decline study sessions so I can manage my time.

```
// ========================
// GET ALL NOTIFICATIONS
// ========================
router.get("/", async (req, res) => {
  try {
    const userId = req.session.user.user_id;

    const [notifications] = await db.query(`
      SELECT * FROM notifications
      WHERE user_id = ?
      ORDER BY created_at DESC
    `, [userId]);

    res.render("notifications", {
      title: "Notifications",
      notifications
    });

  } catch (err) {
    console.error(err);
    res.status(500).send("Error loading notifications");
  }
});
```

12. As a student, I want to receive reminders for study sessions, so I do not forget.

```
// ========================
// GET ALL NOTIFICATIONS
// ========================
router.get("/", async (req, res) => {
  try {
    const userId = req.session.user.user_id;

    const [notifications] = await db.query(`
      SELECT * FROM notifications
      WHERE user_id = ?
      ORDER BY created_at DESC
    `, [userId]);

    res.render("notifications", {
      title: "Notifications",
      notifications
    });

  } catch (err) {
    console.error(err);
    res.status(500).send("Error loading notifications");
  }
});
```

**Communication:**

These stories are about how users communicate with one another.

13. As a student, I should be able to message my study partner to plan sessions together.

[Code Screenshot]

14. As a student, I need to receive notification of messages

[Code Screenshot]

**Security and Privacy:**

15. As a student, I want my data to be protected and secure to feel safe using the platform.

```
const hashedPassword = await bcrypt.hash(password, 10);
```

16. As a student, I should have control over the information that can be visible to other students to protect my privacy.

```
// ADDED — HIDE INFORMATION (POST)
router.post('/hide-info', async (req, res) => {
    const userId = req.session.user.user_id;

    const hide_email = req.body.hide_email ? 1 : 0;
    const hide_degree = req.body.hide_degree ? 1 : 0;
    const hide_modules = req.body.hide_modules ? 1 : 0;
    const hide_picture = req.body.hide_picture ? 1 : 0;

    await req.db.query(
        `INSERT INTO user_privacy (user_id, hide_email, hide_degree, hide_modules, hide_picture)
         VALUES (?, ?, ?, ?, ?)
         ON DUPLICATE KEY UPDATE
         hide_email = VALUES(hide_email),
         hide_degree = VALUES(hide_degree),
         hide_modules = VALUES(hide_modules),
         hide_picture = VALUES(hide_picture)`,
        [userId, hide_email, hide_degree, hide_modules, hide_picture]
    );

    res.redirect('/profile');
});
```

**Maintaining connections**

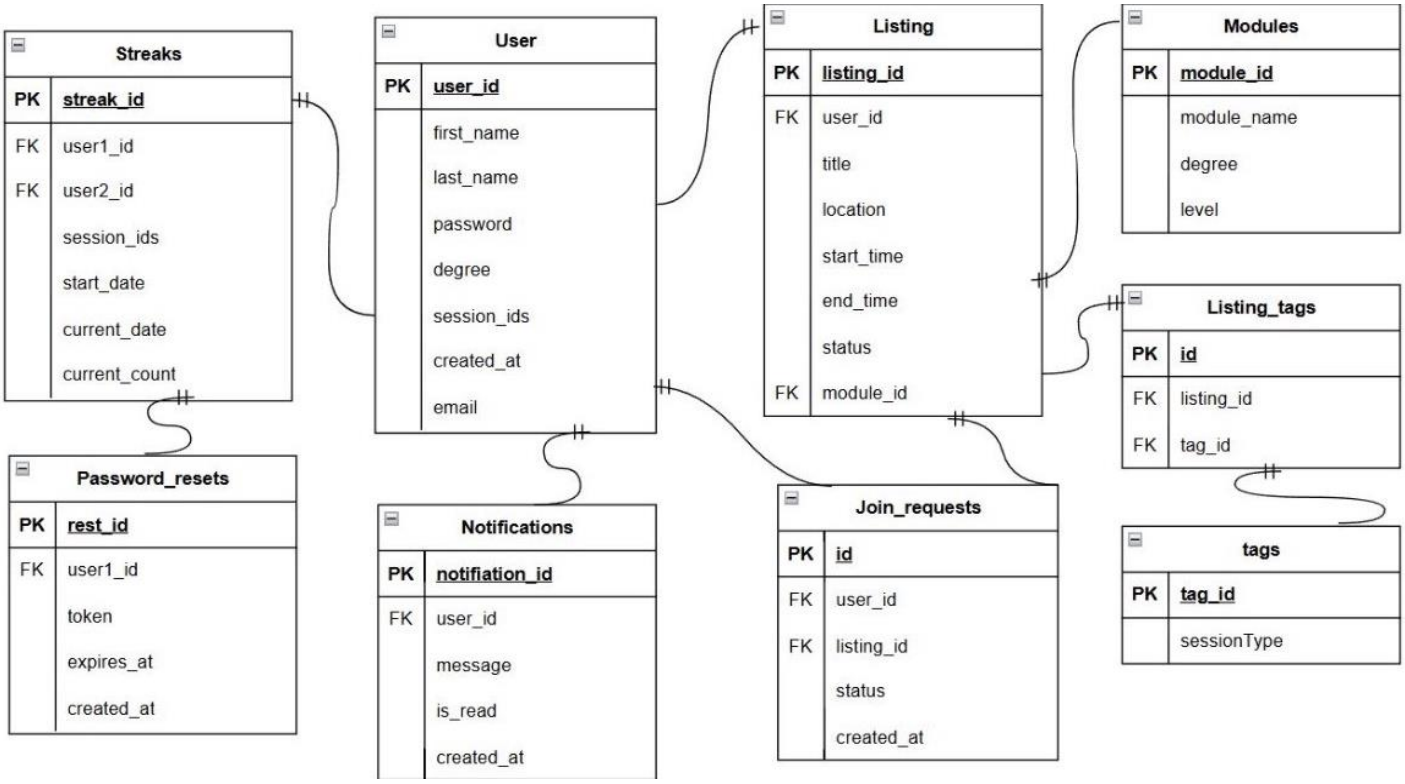
17. As a student I want to view and track the streaks i have with other users

```
let streak = 0;
if (pastSessionDates.length > 0) {
    const today = new Date();
    today.setHours(0, 0, 0, 0);
    let expectedDate = today;

    for (const row of pastSessionDates) {
        const sessionDate = new Date(row.session_date);
        sessionDate.setHours(0, 0, 0, 0);
        if (sessionDate.getTime() === expectedDate.getTime()) {
            streak++;
            expectedDate.setDate(expectedDate.getDate() - 1);
        } else {
            break;
        }
    }
}

res.render("streaks", {
    title: "My Streaks",
    listings: listingsWithParticipants,
    streak: streak
```

## Database Design:




## Breakdown of Tasks and Developers Allocated

To tackle this sprint, we divided our work up with individual members focusing on certain parts of the code, to bring back to the group so we could all test and run it, as well as merge our work

Below are the following responsibilities/ breakdowns of the tasks and corresponding members:

**Database designer and Profile Page Lead** – Sita Panda

- Creating all my SQL tables
- Writing all the Seed SQL data
- Uploading all the database code onto GitHub
- Writing up the ERD diagram
- Creating the model files

**User Routing and PUGs creation Lead –** Richard Akole

- Centralizing all the relevant documents into and SRC file
- Creating all the routes for users
- Creating the PUG file for the User
- Creating PUG file for tags

**Listings and Subject Routing and PUG creation Lead** – Olivia Stars

- Creating all the routes for the listings and session data
- Creating the PUG file for the listings
- Identification and resolution of database
- Troubleshooting docker issues
- Cross-team working on routing issues
- Redesigning the Style CSS file to fit with wireframe

**Authentication.Js and Documentation Lead** – Imogene Garthwaite

- Monitoring the Kanban Board
- Making sure everything is loaded onto GitHub
- Writing up the Sprint 3 Report
- Creating the Login and Register page Pug and routing file
- Creating a forget password page
- Hashing the passwords for security
- Redesigned ERD
- Redesigning task bar and layout.pug
- Implementing a session time out function
- Creating a flowchart to make note of user story functions and routing planning

**Messaging and Notification Lead –** Christy Francis

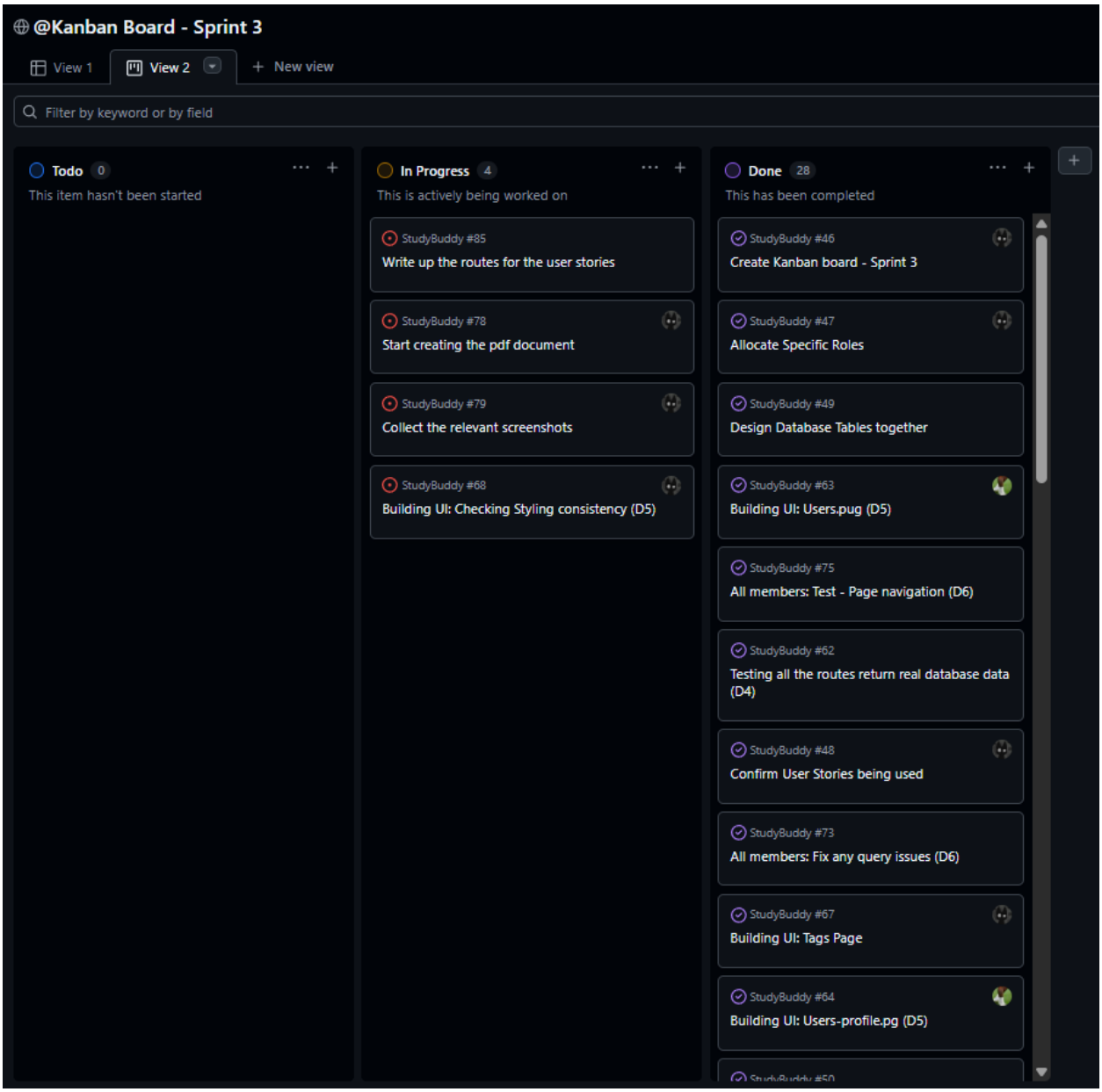
- Writing the minutes of each meeting
- Creating the Routes for each user story
- Creating the messages page
- Updating Schema and Seed Data

**Git Hub repository link:** https://github.com/ImmyGarth01/StudyBuddy.git

**GitHub Project link:** https://github.com/users/ImmyGarth01/projects/6/views/2

**Participation Metrics:**

**Screenshot of the Kanban Board:**


@Kanban Board - Sprint 3
View 1
View 2
New view
Filter by keyword or by field
Todo 0
This item hasn't been started
In Progress 4
This is actively being worked on
StudyBuddy #85
Write up the routes for the user stories
StudyBuddy #78
Start creating the pdf document
StudyBuddy #79
Collect the relevant screenshots
StudyBuddy #68
Building UI: Checking Styling consistency (D5)
Done 28
This has been completed
StudyBuddy #46
Create Kanban board - Sprint 3
StudyBuddy #47
Allocate Specific Roles
StudyBuddy #49
Design Database Tables together
StudyBuddy #63
Building UI: Users.pug (D5)
StudyBuddy #75
All members: Test - Page navigation (D6)
StudyBuddy #62
Testing all the routes return real database data (D4)
StudyBuddy #48
Confirm User Stories being used
StudyBuddy #73
All members: Fix any query issues (D6)
StudyBuddy #67
Building UI: Tags Page
StudyBuddy #64
Building UI: Users-profile.pg (D5)

# Meeting Minutes

| | |
|---|---|
| **Date and Time** | 27/02/2026: 9 - 12<br>06/03/2026: 9 – 12<br>11/03/2026: 9 – 12<br>13/03/2026: 9 – 12<br>20/03/2026: 9 - 12<br>23/03/2026: 9 – 12<br>27/03/2026: 9 – 12 |
| **Project Name** | Study Buddy |
| **Meeting Goals** | • Agree on work flow<br>• Review work that has been done<br>• Plan next steps<br>• Agree on documentation |
| **Facilitator** | Imogene Garthwaite |
| **Note taker** | Imogene Garthwaite |
| **Attendees** | **Sita Panda**<br>**Olivia Stars**<br>**Imogene Garthwaite** |

| | |
|---|---|
| | **Richard Akole**<br><br>**Christy Francis** |
| **Discussion points** | 27/02/2026:<br>• Reviewing the ERD<br>• Planning the database tables<br>06/03/2026:<br>• Highlighting and implementing changes to the database tables<br>• Restructuring and centring into SRC folder<br>• Review of the user features and planning<br>• Managing GitHub issues<br>• Talking through the routing issues and allocation<br>• Docker Troubleshooting<br>11/03/2026:<br>• Managing code structure consistency<br>13/03/2026:<br>• Cleaning up duplicate files<br>• Moving SQL files<br>• Allocating listings, buddies and tags<br>• Bringing awareness to performance metrics on GitHub<br>• Fixing the layout to be more attractive (closer to wireframe)<br>• Docker container troubleshooting<br>• Reinstating the .env file<br>• Updating the ERD<br>• Updating the database<br>20/03/2026:<br>• Changing the layout and style CSS<br>• Updating the seed and scheme files<br>• Working on the login and register pages<br>• Allocating streaks, notifications and messages tasks: pugs and routing<br>• Updating the flowchart<br>23/03/2026:<br>• Restructuring and rerouting to be more reflective of the wireframe and user story<br>• Implementation of authentication hashing meaning all members to register users<br>• Updating the database to reflect changes<br>• Discussion on listing and user routes<br>• Reviewing current documentation<br>27/03/2026:<br>• Discussing routing between notifications and messaging<br>• Discussion of profile features<br>• Implementing successful messaging feature<br>• Updating the ERD<br>• Updating the database<br>• Reviewing of current documentation<br>• Restructuring the layout page |

| | |
|---|---|
| **Actions (list tasks and assign a group member)** | **Sita:**<br>• Creating all my SQL tables<br>• Writing all the Seed SQL data<br>• Uploading all the database code onto GitHub<br>• Writing up the ERD diagram<br>• Creating the model files<br><br>**Richard:**<br>• Centralizing all the relevant documents into and SRC file<br>• Creating all the routes for users<br>• Creating the PUG file for the User<br>• Creating PUG file for tags<br><br>**Imogene:**<br>• Monitoring the Kanban Board<br>• Making sure everything is loaded onto GitHub<br>• Writing up the Sprint 3 Report<br>• Creating the Login and Register page Pug and routing file<br>• Creating a forget password page<br>• Hashing the passwords for security<br>• Redesigned ERD<br>• Redesigning task bar and layout. Pug<br>• Implementing a session time out function<br>• Creating a flowchart to make note of user story functions and routing planning<br><br>**Olivia:**<br>• Creating all the routes for the listings and session data<br>• Creating the PUG file for the listings<br>• Identification and resolution of database<br>• Troubleshooting docker issues<br>• Cross-team working on routing issues<br>• Redesigning the Style CSS file to fit with wireframe<br><br>**Christy:**<br>• Writing the minutes of each meeting<br>• Creating the Routes for each user story<br>• Creating the messages page |